

Project Report

Date	2 October 2026
Team ID	SWTID-2026-6582
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	

1. INTRODUCTION

1.1 Project Overview

PocketSmart AI is a Generative-AI-powered budget and recommendation web application for Home Interior Planning, Party/Event Planning and Jewelry Styling. It combines Google Gemini recommendations with deterministic Python budget validation so the AI can provide contextual ideas while the backend independently verifies arithmetic.

1.2 Purpose

The purpose is to reduce fragmented searching and repeated manual budget calculations by providing one interface that captures user context, generates a structured recommendation plan and clearly shows estimated spending and remaining budget.

2. IDEATION PHASE

2.1 Problem Statement

Users planning interiors, events or jewelry purchases must search across multiple sources, compare many alternatives and manually calculate whether choices fit a fixed budget. Existing keyword search does not fully capture combined context such as room needs, guest count, event services, occasion, outfit color and style.

2.2 Empathy Map Canvas

Users say they have a fixed budget, think about overspending, repeatedly search and compare, and feel overwhelmed by fragmented options. Their main gains are personalized recommendations, transparent estimated budget breakdowns and easier navigation to relevant searches.

2.3 Brainstorming

The team prioritized four groups: core AI planners; modular FastAPI backend; authentication/history/responsive UI; and reliability through budget/image validation and controlled error handling.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey progresses from identifying a planning need, selecting a planner, entering budget/preferences, AI processing, reviewing a validated result, opening search links and returning to saved history.

3.2 Solution Requirement

Functional requirements include authentication, three planners, Gemini integration, deterministic budget validation, recommendation history and search links. Non-functional requirements cover usability, security, reliability, performance, availability and scalability.

3.3 Data Flow Diagram

User input flows through FastAPI/Pydantic to the recommendation orchestrator, Gemini, structured response validation, Python budget calculations, optional SQLite persistence and Jinja2 result rendering.

3.4 Technology Stack

Python, FastAPI, Uvicorn, Google GenAI SDK, Gemini 2.5 Flash Lite, Pydantic, SQLAlchemy, SQLite, Jinja2, HTML/CSS/JavaScript, Pillow, bcrypt, PyJWT and Pytest.

4. PROJECT DESIGN

4.1 Problem Solution Fit

PocketSmart AI directly addresses fragmented search and manual allocation by combining contextual AI generation with deterministic financial validation and optional multimodal styling.

4.2 Proposed Solution

The solution provides three specialized planners, structured Gemini output, a budget revision loop, external search links, authentication and saved recommendation history.

4.3 Solution Architecture

Browser → FastAPI routers → recommendation service → Gemini service → Google Gemini → Pydantic parsing → budget engine → product link generator → SQLite (authenticated users) → Jinja2 result.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The plan uses four sprints covering foundation/authentication, Home and Party planners, Jewelry/history, and final testing/optimization. The documented backlog totals 28 story points.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The automated suite contains 21 tests across seven files. The verified execution result was 21 passed, 0 failed, 0 skipped in approximately 96.44 seconds, including live Gemini and end-to-end workflows.

7. RESULTS

7.1 Output Screenshots

The implemented application provides a landing page, authentication screens, dashboard, Home Planner, Party Planner, Jewelry Planner, budget summary cards, recommendation cards and history. Screenshots should be inserted from the running application for final submission where required by the institute.

8. ADVANTAGES & DISADVANTAGES

Advantages: personalized context-aware planning; three domains in one application; multimodal outfit analysis; deterministic budget calculations; saved history; modular architecture. Limitations: prices are estimates rather than live merchant prices; Gemini requires internet/API quota; LLM outputs can vary; SQLite/local deployment is not intended for large-scale production.

9. CONCLUSION

PocketSmart AI demonstrates a practical hybrid architecture in which Generative AI provides contextual recommendations while conventional backend logic provides validation, authentication, persistence and deterministic calculations. The verified implementation successfully supports Home, Party and Jewelry planning, including multimodal outfit analysis.

10. FUTURE SCOPE

Future work can include verified live marketplace APIs, real-time price/stock comparison, cloud deployment, PostgreSQL, caching, multilingual/voice input, richer user profiles, notifications, wishlists and mobile applications.

11. APPENDIX

Source Code (if any)

Source code is organized under app/, templates/, static/ and tests/. Key services include gemini_service.py, recommendation_service.py, budget_service.py, image_service.py and product_provider.py.

Dataset Link

Not applicable. The implemented project does not use a separate training dataset; it processes user inputs and Gemini API responses at request time.

GitHub & Project Demo Link

To be inserted by the team before final submission. No verified repository or demo-video URL was provided in the supplied project information.